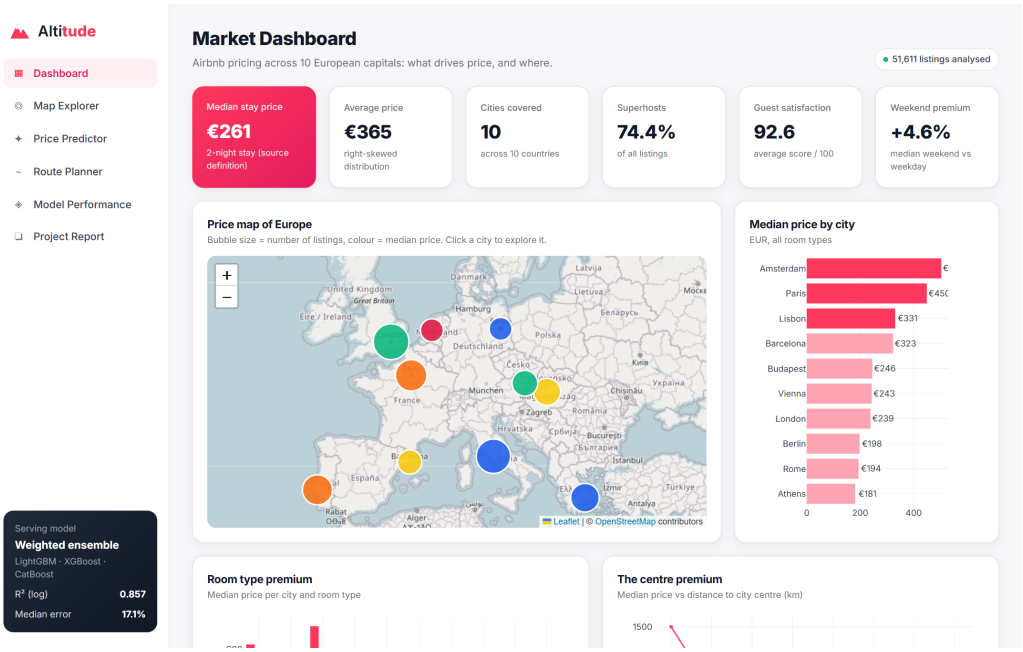
PYTHON PROJECT REPORT

Altitude: Airbnb Price Intelligence in Python

An end-to-end Python application: data cleaning with pandas, analysis with matplotlib, machine learning with scikit-learn and gradient boosting, and a Flask web app with interactive maps



Subject	Python programming
Dataset	51,707 Airbnb listings, 10 European cities, 29 columns
Best model	Weighted ensemble, R^2 0.857, median error 17.1%
Live application	https://airbnb-price-predictor-ivory.vercel.app
Source code	https://github.com/lovishv1718/airbnb-price-predictor
Date	30 October 2026

Table of Contents

1. Introduction 3

2. Data Handling with pandas 4

3. Data Analysis and Visualisation 6

4. Machine Learning in Python 7

5. Object-Oriented Design 9

6. Web Application with Flask 10

7. Testing, Automation and Deployment 14

8. Python Concepts Used 15

9. Conclusion 15

1. Introduction

Altitude is a complete Python application that predicts the price of an Airbnb listing in ten European cities (Amsterdam, Athens, Barcelona, Berlin, Budapest, Lisbon, London, Paris, Rome and Vienna) and lets users explore the market on interactive maps. Every part of it, from the data pipeline to the machine-learning models, the algorithms and the web server, is written in Python.

1.1 Objectives

- Load, validate and clean a real 51,707-row dataset with pandas, documenting every change.
- Analyse the data and produce publication-quality charts with matplotlib and seaborn.
- Train, compare and explain six regression models with scikit-learn, LightGBM, XGBoost and CatBoost.
- Package the model in object-oriented Python and serve it through a Flask web application and REST API.
- Test the code with unittest, automate screenshots with Playwright, and deploy the app publicly.

1.2 Technology stack

Library	Version	Used for
Python	3.13	Language for every component
pandas	2.3.3	Loading, cleaning, grouping and aggregating the data
NumPy	2.3.5	Vectorised maths, haversine distances, the numpy model evaluator
matplotlib / seaborn	3.10.8 / 0.13.2	All report figures
scikit-learn	1.8.0	Ridge, Random Forest, grouped splits, metrics
LightGBM / XGBoost / CatBoost	4.6.0 / 3.0.2 / 1.2.8	Gradient-boosted tree models
SHAP	0.49.1	Model explanations
Flask	3.0.0	Web server, page routes and JSON API
unittest, Playwright, ReportLab	-	Testing, screenshot automation, this PDF

1.3 Project structure

```
altitude/
  app.py                 Flask app: page routes + REST API
  run_pipeline.py        runs every pipeline step with subprocess
  src/config.py          shared paths and constants (pathlib)
  src/clean.py           step 1: cleaning and validation
  src/eda.py             step 2: figures + dashboard data
  src/features.py        feature engineering shared by training and serving
  src/train.py           step 3: training, evaluation, model export
  src/export_lite.py     LightGBM -> numpy arrays for serverless hosting
  src/predictor.py       Predictor and LiteLGBM classes (inference)
  src/algorithms.py      graph algorithms for the Route Planner
  templates/, static/    Jinja2 HTML templates, CSS, JavaScript
  tests/                 unittest test suite
  models/, reports/      trained models, metrics, figures, reports
```

Each step is a separate module with a `main()` function guarded by `if __name__ == "__main__"`, so it can be run on its own or imported. All paths come from one module, `config.py`, built with `pathlib`, so the project runs from any folder.

```
"""Shared paths and constants for the Airbnb price prediction project."""
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
RAW_CSV = ROOT / "Cleaned_data.csv"
DATA_DIR = ROOT / "data"
PROCESSED_CSV = DATA_DIR / "processed.csv"
MODELS_DIR = ROOT / "models"
REPORTS_DIR = ROOT / "reports"
FIG_DIR = ROOT / "static" / "figures"
ARTIFACTS = MODELS_DIR / "artifacts.json"

TARGET = "price_total"
RANDOM_STATE = 42
```

2. Data Handling with pandas

The raw file has 51,707 rows and 29 columns. Each row is one listing on a weekday or a weekend, with its price (a 2-night stay for 2 people, in euros), room type, capacity, host flags, review scores, location measures (distance to the centre and the metro, attraction and restaurant indices) and city-level economic indicators.

2.1 The cleaning function

`clean()` returns the cleaned DataFrame and a log. A nested helper function, `step()`, records the row count before and after each stage. The log is saved as JSON, so the cleaning is fully documented and reproducible.

```
def clean(df: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    log = {"raw_rows": int(len(df)), "raw_cols": int(df.shape[1]), "steps": []}

    def step(name, before, after, note=""):
        log["steps"].append({"step": name, "rows_before": int(before),
                             "rows_after": int(after), "removed": int(before - after),
                             "note": note})

    # 1. Drop the exported index column
    df = df.drop(columns=[c for c in df.columns if c.startswith("Unnamed")])
    step("Drop index column", len(df), len(df), "'Unnamed: 0' was a pandas export index")

    # 2. Missing values
    n = len(df)
    missing = int(df.isna().sum().sum())
    df = df.dropna(subset=[TARGET])
    step("Missing values check", n, len(df), f"{missing} missing cells found")

    # 3. Exact duplicates
    n = len(df)
    df = df.drop_duplicates()
    step("Remove exact duplicates", n, len(df))

    # 4. Fix geocoding errors: some Rome listings were reverse-geocoded to the Vatican
    vatican = int((df["country_name"] != "Italy").where(df["city"] == "Rome", False).sum())
    df.loc[df["city"] == "Rome", "country_name"] = "Italy"
    step("Fix country label for Rome", len(df), len(df),
         f"{vatican} rows labelled 'Holy See (Vatican City State)' -> 'Italy'")

    # 5. Type consistency
    df["max_guests"] = df["max_guests"].round().astype(int)
    df["room_type"] = df["room_type"].str.strip()
    flags_ok = ((df["room_type"] == "Shared room") == (df["is_shared_room"] == 1)) & \
               ((df["room_type"] == "Private room") == (df["is_private_room"] == 1))
    n = len(df)
    df = df[flags_ok]
    step("Room-type flag consistency", n, len(df), "room_type must agree with is_*_room flags")

    ...
```

Listing 2.1 - src/clean.py: the first half of clean().

2.2 Results

Step	Removed	Detail
Drop index column	0	'Unnamed: 0' was a pandas export index
Missing values check	0	0 missing cells found
Remove exact duplicates	63	-
Fix country label for Rome	0	4580 rows labelled 'Holy See (Vatican City State)' -> 'Italy'
Room-type flag consistency	0	room_type must agree with is_*_room flags
Domain range checks	0	-
Remove extreme price outliers	33	per-city 3xIQR fence on log(price); removes data-entry errors such as 43,955 EUR

Several pandas idioms do the work. `drop_duplicates()` removes 63 copies, a boolean `.loc` assignment fixes 4,580 Rome rows mislabelled as the Vatican, and `groupby(...).transform()` computes per-city quartiles for the outlier fence without a Python loop. `groupby(...).ngroup()` gives every listing an id. That revealed that 18,774 listings appear twice, once on a weekday and once on a weekend, which matters

for honest model evaluation (Chapter 4).

3. Data Analysis and Visualisation

`eda.py` turns the cleaned data into figures and a JSON summary for the dashboard. The median stay costs €261 and the mean €365. Prices are right-skewed, so the models predict log price.

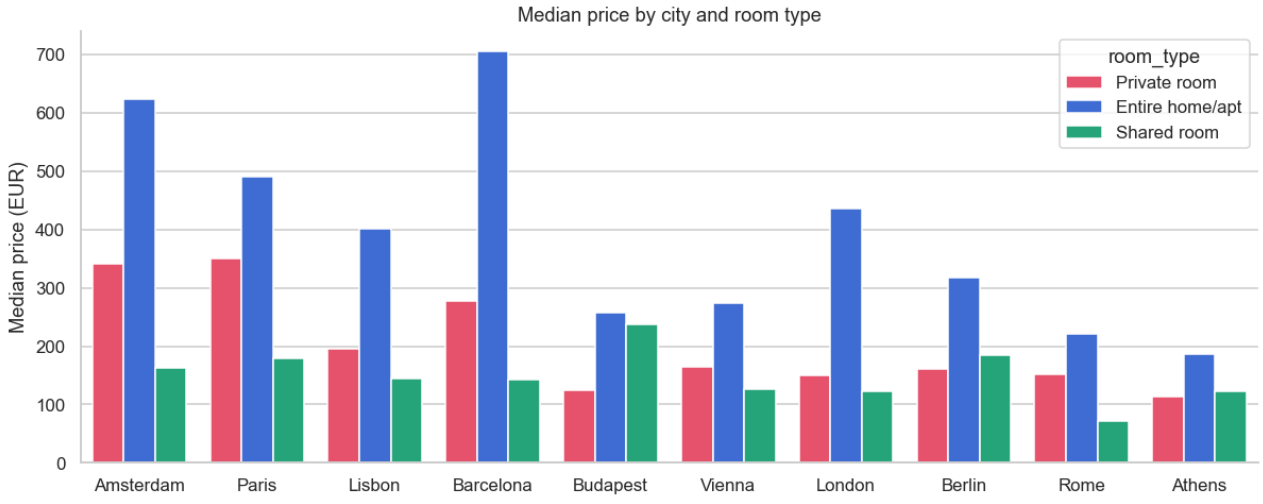


Figure 3.1 - Median price by city and room type (seaborn barplot).

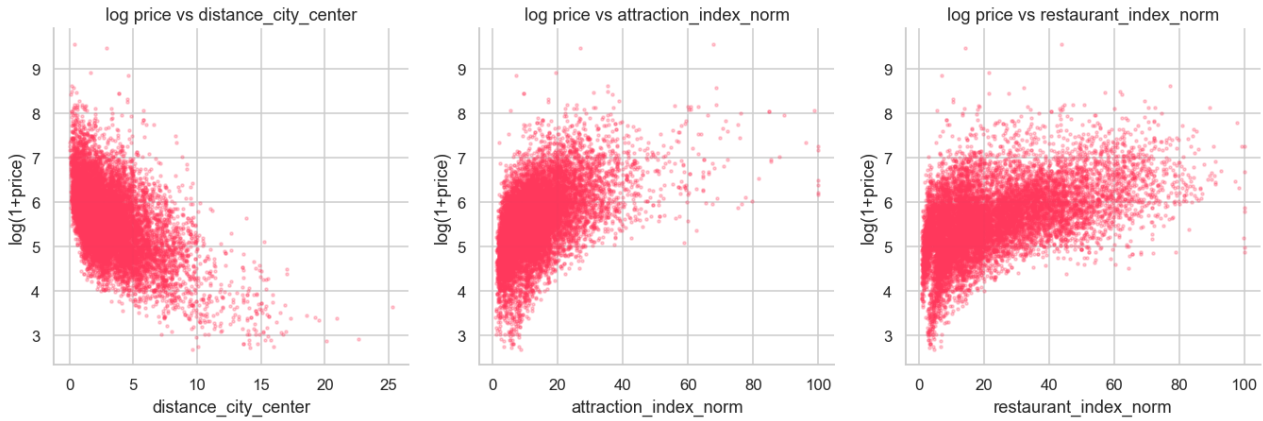


Figure 3.2 - Log price against distance to centre, attraction and restaurant indices (matplotlib scatter).

Location is the strongest signal: price falls steeply in the first 1 to 2 km from the centre and rises with the attraction and restaurant indices. Amsterdam and Paris are the most expensive cities and Athens the cheapest. Superhost status and reviews make little difference to price.

```
def save(fig, name):
    fig.savefig(FIG_DIR / f"{name}.png")
    plt.close(fig)
```

Listing 3.1 - A small helper keeps every figure consistent; the global style is set once with sns.set_theme() and plt.rcParams.

The dashboard summary is built with `groupby`, `pivot_table`, `pd.cut` for distance bands and `np.histogram`, then written with `json.dumps`. The web app reads this file at start-up instead of recomputing it on every request.

4. Machine Learning in Python

4.1 Feature engineering

One function, used by both training and the web app, adds six engineered features. Because it is shared, the features are always computed exactly the same way in both places.

```
def base_features(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["is_weekend"] = (out["day_type"] == "weekend").astype(int)
    out["guests_per_bedroom"] = out["max_guests"] / out["num_bedrooms"].clip(lower=1)
    out["log_dist_center"] = np.log1p(out["distance_city_center"])
    out["log_dist_metro"] = np.log1p(out["distance_metro"])
    out["attr_x_rest"] = out["attraction_index_norm"] * out["restaurant_index_norm"] / 100
    out["quality_score"] = out["cleanliness_score"] * 10 * 0.4 + out["guest_satisfaction_score"] * 0.6
    for c in CATEGORICAL:
        out[c] = out[c].astype("category")
    return out
```

Listing 4.1 - src/features.py: vectorised pandas feature engineering.

4.2 Leakage-free evaluation

`GroupShuffleSplit` from scikit-learn holds out 20% of listings (10,366 rows), grouped by `listing_id`, so the weekday and weekend copies of a listing are never split between training and test. A random split would report R^2 0.876; the honest grouped score is 0.857.

```
def metrics(y_true_log, y_pred_log):
    yt, yp = np.expm1(y_true_log), np.expm1(y_pred_log)
    ape = np.abs(yp - yt) / yt
    return {
        "r2_log": round(float(r2_score(y_true_log, y_pred_log)), 4),
        "r2_eur": round(float(r2_score(yt, yp)), 4),
        "mae_eur": round(float(mean_absolute_error(yt, yp)), 2),
        "rmse_eur": round(float(np.sqrt(mean_squared_error(yt, yp))), 2),
        "mape_pct": round(float(ape.mean() * 100), 2),
        "median_ape_pct": round(float(np.median(ape) * 100), 2),
        "within_20pct": round(float((ape <= .2).mean() * 100), 2),
        "within_30pct": round(float((ape <= .3).mean() * 100), 2),
    }
```

Listing 4.2 - src/train.py: all evaluation metrics in one function returning a dictionary.

4.3 Models and results

Model	R^2 log	MAE €	Median error	Within ±30%
City median (baseline)	0.1572	191.4	42.2%	37.2%
Ridge regression	0.8108	99.6	20.7%	67.0%
Random Forest	0.8469	90.1	17.2%	73.1%
LightGBM	0.8542	88.1	17.5%	74.1%
XGBoost	0.8553	87.6	17.1%	74.2%
CatBoost	0.8558	87.3	17.3%	74.2%

Model	R^2 log	MAE €	Median error	Within ±30%
Weighted ensemble	0.8569	87.1	17.1%	74.4%

Six models were compared: a city-median baseline, Ridge regression, Random Forest, LightGBM, XGBoost and CatBoost. The three boosting models use early stopping on a grouped validation fold. The final ensemble averages them with weights found by `scipy.optimize.minimize` (LightGBM 17%, XGBoost 42%, CatBoost 42%). It explains 85.7% of the variation in log price on unseen listings, with a typical error of 17%.

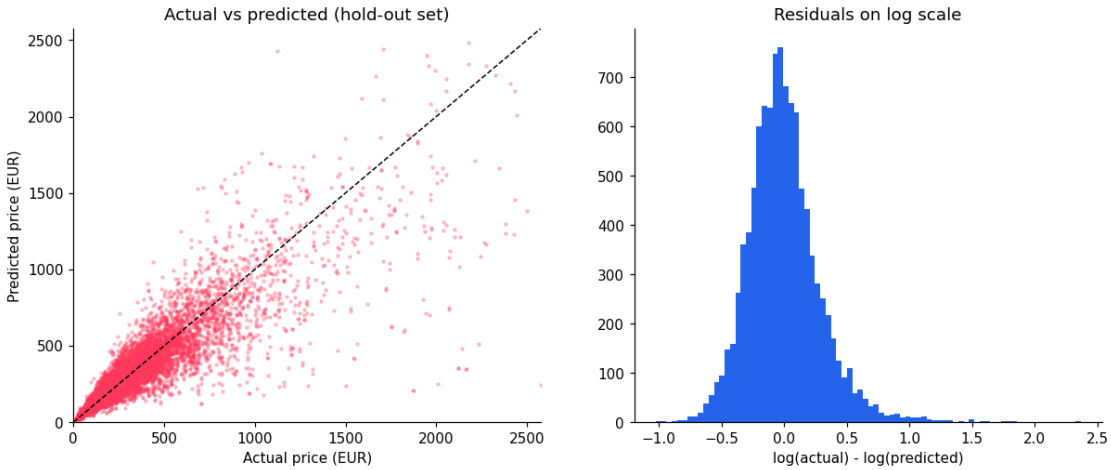


Figure 4.1 - Predicted against actual price on the hold-out set.

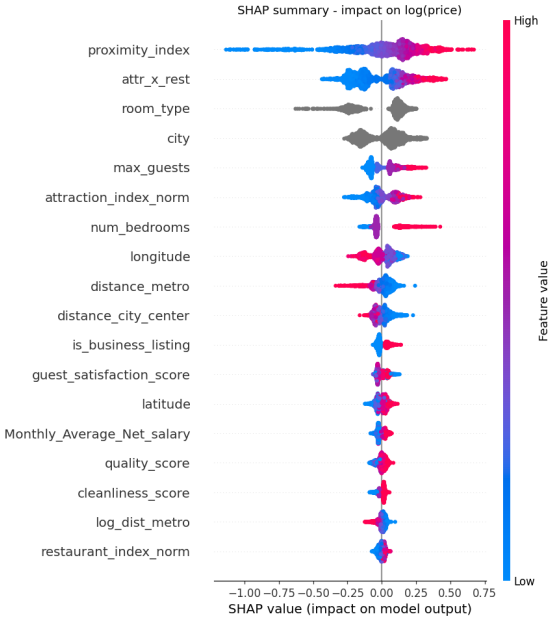


Figure 4.2 - SHAP summary: how each feature moves the price.

The most important features describe how central a location is (proximity, attractions and restaurants), followed by city, room type and capacity.

5. Object-Oriented Design

Inference lives in two classes in `src/predictor.py`. `Predictor` loads everything once when the app starts and exposes one public method, `predict(request_dict)`. It hides where the model comes from:

- **Full mode:** loads the ensemble with joblib when the ML libraries are installed.
- **Lite mode:** if importing them raises `ImportError` (as on Vercel), it falls back to `LiteLGBM`, which evaluates the exported LightGBM trees with numpy alone. Its predictions match LightGBM exactly.

```
class LiteLGBM:
    """Numpy-only evaluator for the exported LightGBM trees (see export_lite.py)."""

    def __init__(self, path):
        z = np.load(path)
        self.__dict__.update({k: z[k] for k in z.files})
        self.cat_sets = {int(i): set(self.cats[self.cat_ptr[i]:self.cat_ptr[i + 1]].tolist())
                         for i in np.flatnonzero(self.is_cat)}

    def predict_contrib(self, x):
        """Return (prediction, per-feature contributions) using path attribution."""
        contrib = np.zeros(len(x))
        pred = 0.0
        feature, thr, left, right, value = self.feature, self.threshold, self.left, self.right, self.value
        for n in self.roots.tolist():
            while left[n] != -1:
                f = feature[n]
                go_left = (int(x[f]) in self.cat_sets[n]) if self.is_cat[n] else x[f] <= thr[n]
                child = left[n] if go_left else right[n]
                contrib[f] += value[child] - value[n]
                n = child
            pred += value[n]
        return pred, contrib
```

Listing 5.1 - src/predictor.py: the LiteLGBM class walks each decision tree and accumulates path contributions for the explanation.

A user only gives a map location, so `location_features()` derives the rest. It computes vectorised haversine distances to every listing in the city, sorts them with `np.argsort`, and takes a distance-weighted average of the 25 nearest:

```
def location_features(self, city, lat, lon, k=25):
    d = self.city_data[city]
    dist_km = haversine_km(lat, lon, d.latitude.values, d.longitude.values)
    idx = np.argsort(dist_km, kind="stable")[:k]
    dist_km = dist_km[idx]
    nb = d.iloc[idx]
    w = 1.0 / (dist_km + 0.3)
    feats = {c: float(np.average(nb[c], weights=w)) for c in LOCAL_COLS}
    c_lat, c_lon = self.centers[city]
    feats["distance_city_center"] = float(haversine_km(lat, lon, c_lat, c_lon))
    return feats, nb, float(dist_km.mean())
```

Listing 5.2 - Deriving location features from the nearest real listings.

6. Web Application with Flask

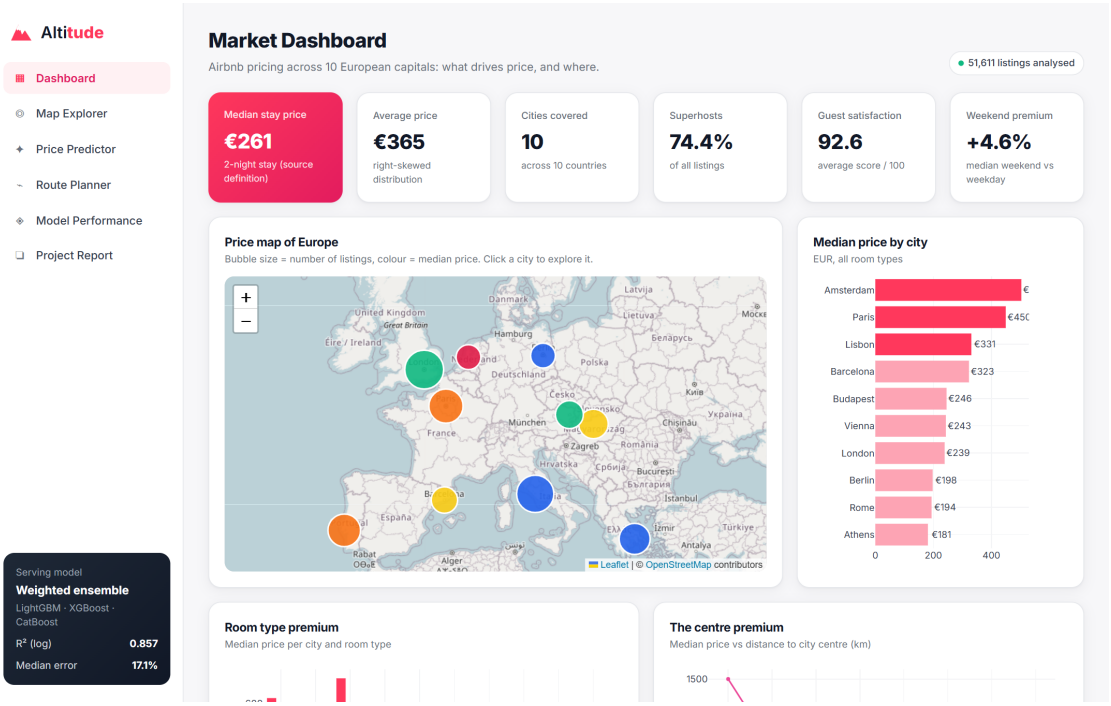
`app.py` creates the Flask app, builds one `Predictor` at start-up and defines six page routes plus a JSON API. Pages are Jinja2 templates that extend a shared `base.html`; a context processor injects the city list and model metrics into every page.

Route	Method	Purpose
/	GET	Market dashboard
/explorer	GET	Map of every listing
/predict	GET	Price predictor
/route	GET	Route Planner (graph algorithms)
/model	GET	Model performance
/report	GET	Report rendered from Markdown
/api/listings/<city>	GET	Listings of one city as compact JSON
/api/predict	POST	Price prediction with explanation
/api/route	POST	Shortest paths and TSP tours
/api/health	GET	Status check

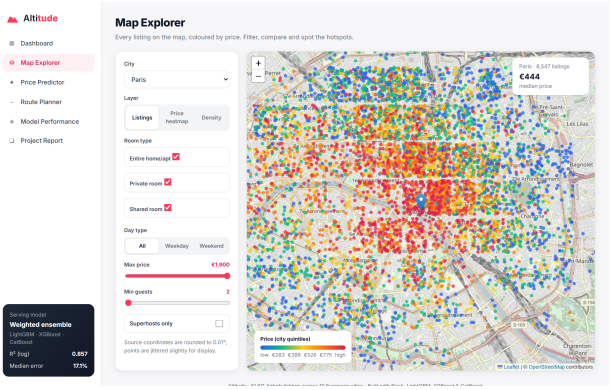
```
def api_predict():
    req = request.get_json(force=True)
    if req.get("city") not in predictor.city_data:
        return jsonify({"error": "Unknown city"}), 400
    try:
        return jsonify(predictor.predict(req))
    except (ValueError, KeyError) as e:
        return jsonify({"error": str(e)}), 400
```

Listing 6.1 - app.py: a JSON endpoint with input validation and error handling.

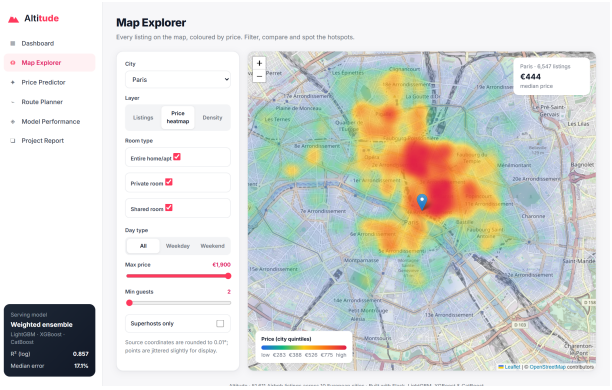
6.1 Dashboard and map explorer



Screenshot 6.1 - Dashboard: KPI cards, Europe price map, median price by city.



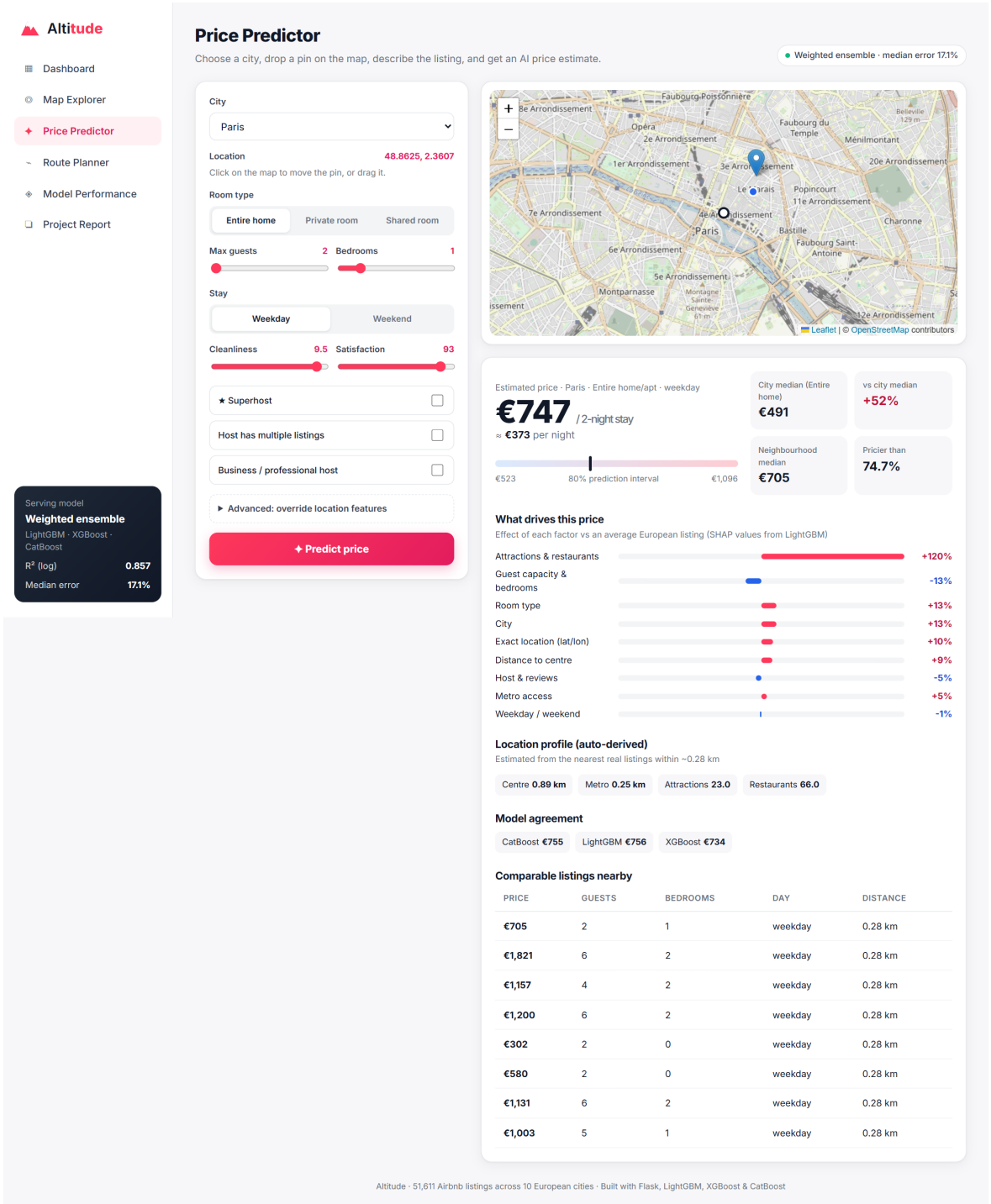
Listings layer



Price heatmap layer

Screenshot 6.2 - Map Explorer, Paris: every listing coloured by price, and the price heatmap.

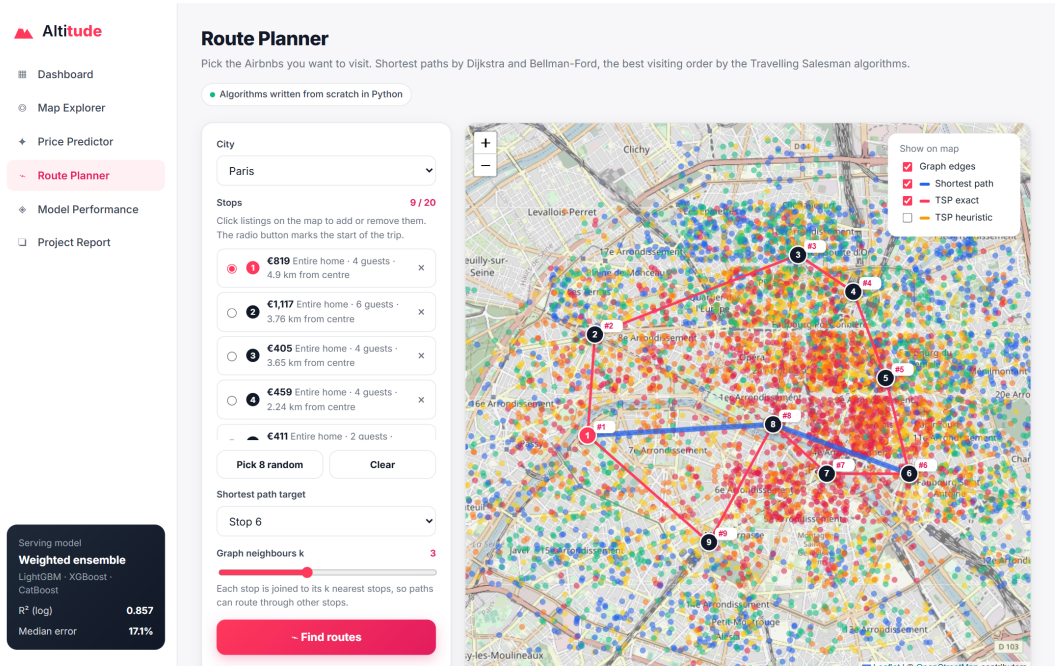
6.2 Price predictor



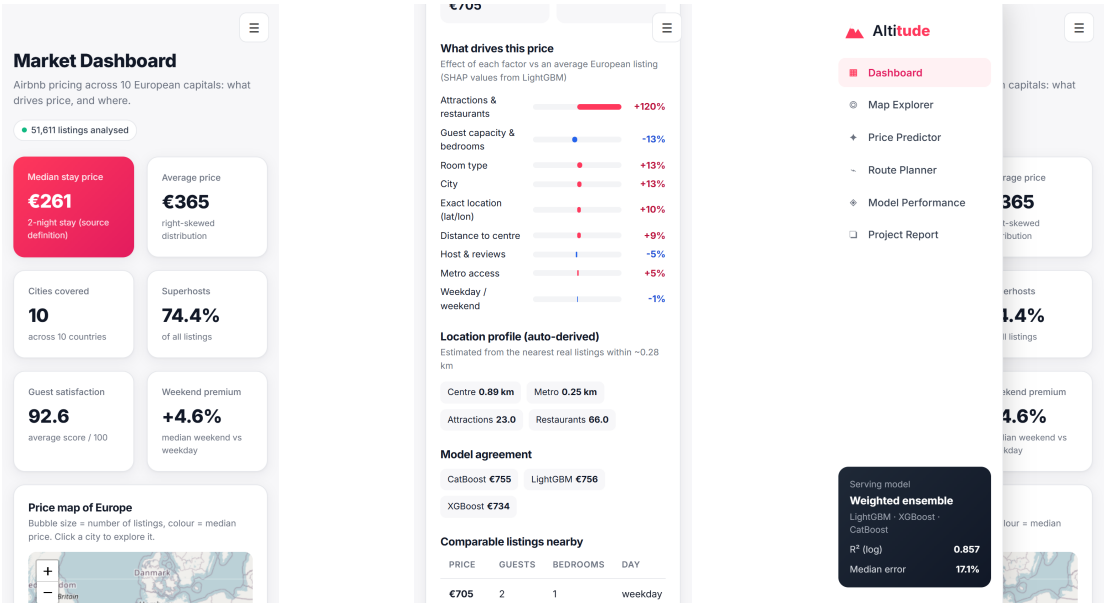
Screenshot 6.3 - Prediction for a central Paris apartment: price, interval, drivers, model agreement and comparable listings.

The browser sends the form as JSON with `fetch()`. Flask passes it to `Predictor.predict()`, and `jsonify()` returns the result. The page then draws the drivers chart and the comparable listings on the Leaflet map.

6.3 Route Planner and responsive design



Screenshot 6.4 - Route Planner: stops chosen on the map, optimal tour and shortest path (the algorithms are covered in the DAA report).



Dashboard *Prediction result* *Navigation menu*

Screenshot 6.5 - Mobile views (390 × 844).

7. Testing, Automation and Deployment

7.1 Unit tests

`tests/test_algorithms.py` uses Python's built-in `unittest` framework: 11 tests in 4 `TestCase` classes, all passing. Each algorithm is checked against an independent reference implementation.

```
def test_hand_worked_example(self):
    # A-B 4, A-C 1, C-B 2, B-D 1, C-D 5  ->  A to D = A-C-B-D = 4
    g = {0: {1: 4, 2: 1}, 1: {0: 4, 2: 2, 3: 1}, 2: {0: 1, 1: 2, 3: 5}, 3: {1: 1, 2: 5}}
    for algo in (dijkstra, bellman_ford):
        dist, prev, _ = algo(g, 0)
        self.assertEqual(dist[3], 4)
        self.assertEqual(path_to(prev, 3), [0, 2, 1, 3])
```

Listing 7.1 - A unittest test case.

7.2 Other checks

Check	How	Result
No listing in both train and test	set intersection of listing ids	0 overlap
Feature leakage	rebuild proximity_index with and without price	none
Lite model fidelity	numpy evaluator vs LightGBM on 300 rows	max difference 0.0
Pages and API	HTTP requests locally and on Vercel	all HTTP 200
UI interactions	Playwright scripts (capture_screenshots.py)	working

The screenshots in this report are captured automatically. `capture_screenshots.py` drives a real Chrome browser with Playwright: it opens each page, clicks the layer buttons, fills the predictor form and saves the images, so they can be regenerated after any change.

7.3 Deployment

The code is on GitHub and the app runs on Vercel's serverless Python runtime. `requirements.txt` lists only Flask, numpy, pandas and markdown, which keeps the bundle under Vercel's size limit. `requirements-ml.txt` has the full training stack. When the ML libraries are missing, `Predictor` switches to the numpy model automatically (Chapter 5).

8. Python Concepts Used

Concept	Where it is used
Functions, default arguments, return of tuples	clean() returns (df, log); plan_routes(points, start=0, k=3)
Nested functions and closures	step() inside clean(); connect() inside build_graph()
Lambdas	loss function in ensemble_weights(); sort keys in tsp_nearest_neighbour()
Classes, __init__, methods, state	Predictor, LiteLGBM, ReportDoc (inherits BaseDocTemplate)
Inheritance and method overriding	ReportDoc overrides afterFlowable() for the table of contents
Exceptions	try/except ImportError (lite mode); ValueError/KeyError in API handlers
Modules, packages, __name__ guard	every src/*.py module; shared config.py
Comprehensions and generators	dict and list comprehensions throughout; any(...) negative-cycle check
Standard library	pathlib, json, heapq, itertools, math, time, random, subprocess, unittest
File I/O	CSV with pandas, JSON reports, joblib/npz models, PNG figures, PDF
Data structures	dicts as graphs, sets for visited nodes, heaps, bitmask integers
Vectorisation	pandas groupby/transform, NumPy haversine and argsort
Decorators	Flask @app.route and @app.context_processor
Testing	unittest.TestCase, assertAlmostEqual, randomised tests

9. Conclusion

Altitude shows how far a single language can take a project, from raw CSV to a live web product. pandas handled the cleaning, matplotlib and seaborn the analysis, and scikit-learn with three gradient-boosting libraries the modelling. The result explains 85.7% of the variation in log price on unseen listings. Object-oriented design keeps the serving code clean and lets it run with or without the heavy libraries. Flask exposes it all through six pages and a JSON API, and unittest and Playwright check that it works.

Limitations and future work

- Photos, floor area and amenities are not in the data and probably explain most of the remaining error.
- The data is one snapshot, with no seasonality.
- Possible next steps: type hints and mypy, pytest with coverage, Docker packaging, and a FastAPI version of the API.

References

- Python Software Foundation. Python 3 documentation, https://docs.python.org/3/
- McKinney, W. (2022). *Python for Data Analysis*, 3rd ed. O'Reilly.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825-2830.
- Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS* 30.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS* 30.
- Gyódi, K. and Nawaro, Ł. (2021). Determinants of Airbnb prices in European cities. *Tourism Management*, 86.
- Flask documentation, https://flask.palletsprojects.com